

TERM-II PROJECT REPORT

Personal Investment Portfolio Tracker

Object-Oriented System Design & Implementation

Course: Object-Oriented Programming

Student Name: Ayush Das

Roll Number: 37

Enrolment Number: 12024052017009

Instructor: Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas

Term: Spring 2026

1. Problem Statement

Individual investors managing personal portfolios across multiple asset types ; stocks, mutual funds, and bonds; often rely on disconnected spreadsheets or manual tracking, which lack real-time recalculation and proactive alerts. This leads to poor visibility into actual returns and missed threshold-based decisions. The Personal Investment Portfolio Tracker addresses this by providing a structured, object-oriented system that models each investment type distinctly, calculates returns polymorphically, persists portfolio state across sessions, and notifies users when gain or loss thresholds are breached ; enabling disciplined, informed portfolio management.

2. System Design

2.1 UML Class Diagram

The diagram below illustrates the full class hierarchy and relationships. Hollow triangle arrows denote inheritance; dashed arrows denote interface implementation; solid diamond arrows denote composition/dependency.

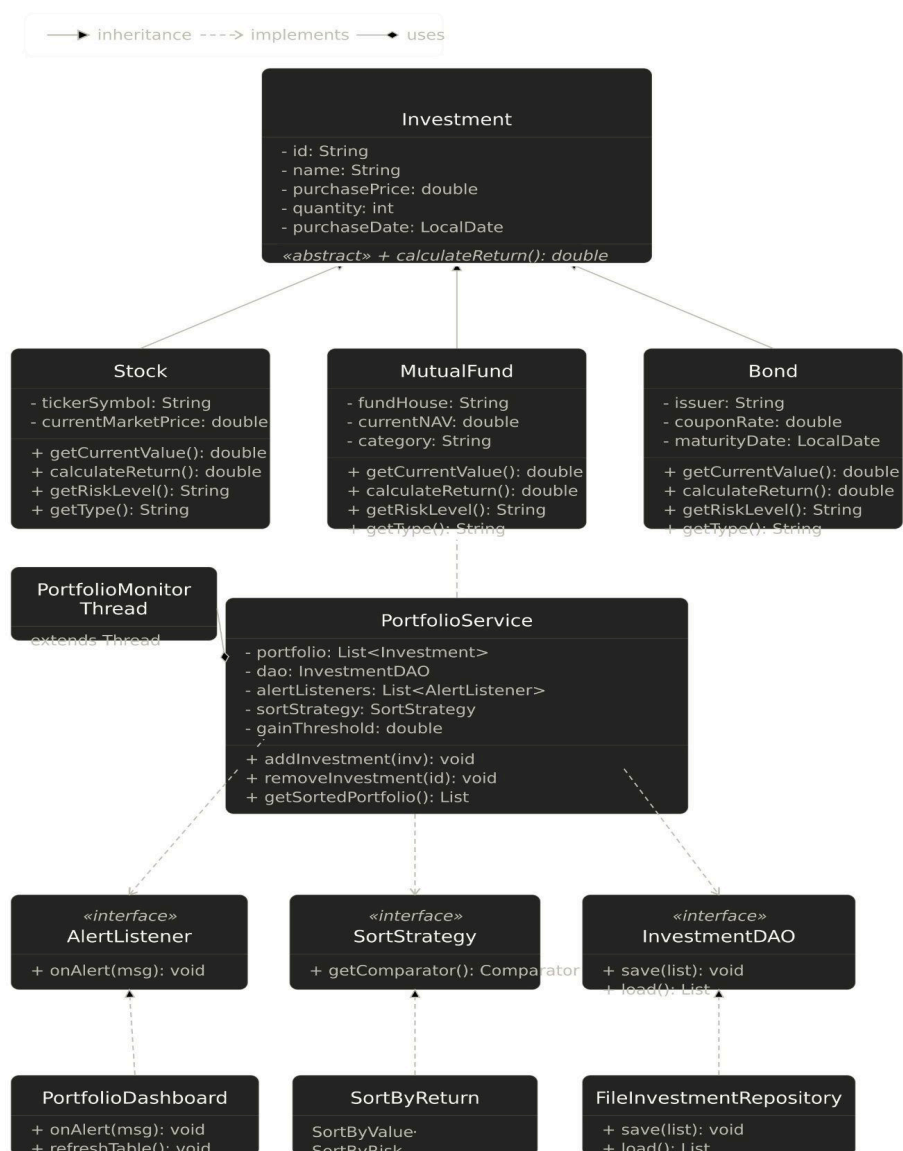


Figure 1: UML Class Diagram — Personal Investment Portfolio Tracker

2.2 Description of Major Classes

The system is organised into six packages, each with a clearly defined responsibility. The following table describes the key classes and their roles:

Class	Package	Responsibility
Investment	model	Abstract base class. Defines shared attributes (id, name, purchasePrice, quantity, purchaseDate) and enforces abstract methods calculateReturn(), getCurrentValue(), getRiskLevel(), getType() on all subclasses.
Stock	model	Extends Investment. Computes return based on current market price vs purchase price. Risk level determined by return percentage magnitude.
MutualFund	model	Extends Investment. Computes return based on NAV (Net Asset Value) change. Risk level determined by fund category (Equity/Hybrid/Debt).
Bond	model	Extends Investment. Computes return based on coupon rate and holding period in years. Overrides getReturnPercentage() to use coupon-based logic.
PortfolioService	service	Central business logic coordinator. Manages CRUD operations, validation, sorting via Strategy pattern, threshold evaluation via Observer, and delegates persistence to DAO.
PortfolioMonitorThread	service	Daemon thread extending Thread. Periodically invokes PortfolioService.checkAllThresholds() without blocking the UI event dispatch thread.
AlertListener	service	Observer interface. Defines onAlert(String message). Implemented by PortfolioDashboard to receive threshold breach notifications.
SortStrategy	service	Strategy interface. Defines getComparator(). Implemented by SortByReturn, SortByValue, and SortByRisk — swappable at runtime.
InvestmentDAO	repository	DAO interface. Decouples service layer from storage implementation. Defines save(List) and load() contract.
FileInvestmentRepository	repository	Implements InvestmentDAO. Serialises portfolio to a .dat file using ObjectOutputStream and deserialises on load.

Class	Package	Responsibility
InvalidInvestmentException	exception	Custom checked exception. Thrown when investment input fails validation — empty name, non-positive price, or zero quantity.
PortfolioPersistenceException	exception	Custom checked exception. Thrown when file I/O operations fail during save or load.
PortfolioDashboard	ui	Main Swing JFrame. Implements ActionListener. Hosts portfolio summary cards, sortable holdings table, and delegates all business operations to PortfolioService.
IdGenerator	util	Utility class. Generates unique, thread-safe investment IDs using AtomicInteger with a type prefix (STK-, MF-, BND-).

Table 1: Major Classes and Their Responsibilities

2.3 Package Structure

Package	Contents
model/	Investment (abstract), Stock, MutualFund, Bond
service/	PortfolioService, PortfolioMonitorThread, ActionListener, SortStrategy, SortByReturn, SortByValue, SortByRisk
repository/	InvestmentDAO (interface), FileInvestmentRepository
exception/	InvalidInvestmentException, PortfolioPersistenceException
ui/	PortfolioDashboard, AddInvestmentDialog, EditInvestmentDialog
util/	IdGenerator

Table 2: Package Structure

2.4 Interaction Flow

The following sequence describes the flow when a user adds a new investment and the system evaluates threshold alerts:

- User fills the Add Investment dialog in the Presentation Layer (PortfolioDashboard) and clicks Add.
- AddInvestmentDialog parses and validates field types locally (NumberFormatException, DateTimeParseException), then constructs the appropriate subclass — Stock, MutualFund, or Bond.
- PortfolioDashboard calls PortfolioService.addInvestment(inv).
- PortfolioService performs domain validation (InvalidInvestmentException for empty name, non-positive values, null date). If valid, the investment is added to the internal List<Investment>.

- `PortfolioService.checkThresholds(inv)` is invoked. If the return percentage breaches the configured gain or loss threshold, all registered `AlertListeners` are notified via `fireAlert(message)`.
- `PortfolioDashboard.onAlert(message)` receives the notification on the Event Dispatch Thread via `SwingUtilities.invokeLater`, updating the alert banner.
- `PortfolioDashboard` refreshes the sorted table (`getSortedPortfolio()`) and recalculates the summary panel values.
- On application close, `PortfolioService.savePortfolio()` delegates to `FileInvestmentRepository.save(list)`, serialising state to `data/portfolio.dat`.

3. OOP Concepts Demonstrated

3.1 Abstraction

Where it is used: The Investment class in the model package.

Why it is used: No real-world entity is simply an 'Investment' — every investment is always a Stock, MutualFund, or Bond. The abstract class forces all subclasses to provide their own implementations of asset-specific behaviour while sharing the common attribute structure (id, name, purchasePrice, quantity, purchaseDate). This ensures consistency across the type hierarchy and prevents instantiation of an incomplete, conceptual type.

Code Snippet:

```
public abstract class Investment implements Serializable,
Comparable<Investment> {
    private String id;
    private String name;
    private double purchasePrice;
    private int quantity;
    private LocalDate purchaseDate;

    // Shared concrete behaviour
    public double getTotalInvested() {
        return purchasePrice * quantity;
    }

    // Contract enforced on all subclasses
    public abstract double getCurrentValue();
    public abstract double calculateReturn();
    public abstract String getRiskLevel();
    public abstract String getType();
}
```

Explanation: The abstract Investment class defines shared state and enforces a behavioural contract. Subclasses must provide implementations for calculateReturn(), getCurrentValue(), getRiskLevel(), and getType(). The class cannot be instantiated directly, ensuring that only fully-defined asset types (Stock, MutualFund, Bond) can exist in the portfolio.

3.2 Inheritance

Where it is used: Stock, MutualFund, and Bond extend Investment.

Why it is used: All three investment types share a significant portion of their structure (identity, pricing, quantity, date). Inheritance eliminates duplication of these common attributes while allowing each subclass to introduce its own domain-specific fields and logic — tickerSymbol and currentMarketPrice for Stock, fundHouse and currentNAV for MutualFund, and couponRate and maturityDate for Bond.

Code Snippet:

```
public class MutualFund extends Investment {
    private String fundHouse;
    private double currentNAV;
    private String category; // Equity, Debt, Hybrid

    public MutualFund(String id, String name, String fundHouse,
                      String category, double purchaseNAV,
                      double currentNAV, int units, LocalDate date) {
        super(id, name, purchaseNAV, units, date); // delegates to Investment
        this.fundHouse = fundHouse;
        this.currentNAV = currentNAV;
        this.category = category;
    }
}
```

```
}

@Override
public double getCurrentValue() { return currentNAV * getQuantity(); }
}
```

Explanation: `MutualFund` calls `super()` in its constructor to initialise the shared `Investment` attributes, then declares only the fields specific to mutual funds. The `@Override` annotation makes the contract implementation explicit. `getQuantity()` is inherited from `Investment` without any code duplication.

3.3 Polymorphism

Where it is used: `PortfolioService` operates on `List<Investment>` and calls `calculateReturn()` on each element.

Why it is used: Return calculation logic differs fundamentally across asset types — `Stock` uses market price movement, `MutualFund` uses NAV change, and `Bond` uses coupon rate over holding period. Polymorphism allows `PortfolioService` to compute the total portfolio return across all holdings using a single loop, without any instanceof checks or type-based branching. Adding a new asset type (e.g. `ETF`) would require only a new subclass — the service layer is unmodified (Open/Closed Principle).

Code Snippet:

```
// In PortfolioService — no type checking, no branching
public double getTotalReturn() {
    return portfolio.stream()
        .mapToDouble(Investment::calculateReturn)
        .sum();
}

// Each subclass overrides calculateReturn() differently:
// Stock:      return (currentMarketPrice - purchasePrice) * quantity;
// MutualFund: return (currentNAV - purchaseNAV) * units;
// Bond:       return totalInvested * couponRate/100 * yearsHeld;
```

Explanation: The `stream().mapToDouble(Investment::calculateReturn)` call invokes the correct subclass implementation at runtime for each element in the list. This is runtime polymorphism — the JVM dispatches the call to the appropriate overridden method based on the actual object type, not the declared `List<Investment>` reference type.

3.4 Exception Handling

Where it is used: `PortfolioService` (`InvalidInvestmentException`) and `FileInvestmentRepository` (`PortfolioPersistenceException`).

Why it is used: Generic exceptions (`IllegalArgumentException`, `IOException`) give callers no structured way to distinguish between a bad investment input and a storage failure. Custom checked exceptions force callers to handle both categories explicitly and provide meaningful, domain-specific error messages to the user interface.

Code Snippet:

```
private void validateInvestment(Investment inv)
    throws InvalidInvestmentException {
    if (inv == null)
        throw new InvalidInvestmentException("Investment cannot be null.");
    if (inv.getName() == null || inv.getName().isBlank())
        throw new InvalidInvestmentException("Name cannot be empty.");
    if (inv.getPurchasePrice() <= 0)
        throw new InvalidInvestmentException(
            "Purchase price must be greater than zero.");
}
```

```
if (inv.getQuantity() <= 0)
    throw new InvalidInvestmentException(
        "Quantity must be greater than zero.");
}
```

Explanation: Each validation rule maps to a specific, human-readable message. The method is private — called internally by `addInvestment()` and `updateInvestment()` before any mutation of the portfolio list. The UI layer catches `InvalidInvestmentException` and displays the message in a `JOptionPane` error dialog, keeping the validation logic entirely within the service layer.

3.5 Collections

Where it is used: `PortfolioService` maintains a `List<Investment>`; sorting is performed via `Comparator` and `Collections.sort()`.

Why it is used: `List` provides $O(1)$ indexed access and maintains insertion order. The `Comparator`-based `Strategy` pattern enables the sort criterion to be swapped at runtime without modifying `PortfolioService` — the active `SortStrategy` is a field that can be replaced via `setSortStrategy()`.

Code Snippet:

```
// Strategy interface
public interface SortStrategy {
    Comparator<Investment> getComparator();
}

// Runtime sort in PortfolioService
public List<Investment> getSortedPortfolio() {
    List<Investment> sorted = new ArrayList<>(portfolio);
    Collections.sort(sorted, sortStrategy.getComparator());
    return sorted;
}

// Concrete strategy - sort by return %
public class SortByReturn implements SortStrategy {
    public Comparator<Investment> getComparator() {
        return Comparator.comparingDouble(
            Investment::getReturnPercentage).reversed();
    }
}
```

Explanation: `getSortedPortfolio()` copies the portfolio list before sorting to preserve the original insertion order. The active `Comparator` comes from whichever `SortStrategy` is currently set. The UI combo box maps its selection to `setSortStrategy(new SortByReturn())`, `setSortStrategy(new SortByValue())`, or `setSortStrategy(new SortByRisk())` — demonstrating runtime strategy substitution.

3.6 Threads

Where it is used: `PortfolioMonitorThread` extends `Thread` and runs as a background daemon.

Why it is used: Threshold evaluation must happen periodically without blocking the Swing Event Dispatch Thread. A daemon thread ensures it terminates automatically when the application closes. The volatile running flag allows clean thread shutdown via `stopMonitor()`.

Code Snippet:

```
public class PortfolioMonitorThread extends Thread {
    private final PortfolioService portfolioService;
    private volatile boolean running = true;
    private static final int INTERVAL_MS = 30_000;

    public PortfolioMonitorThread(PortfolioService svc) {
```

```
        this.portfolioService = svc;
        setDaemon(true);    // dies when main app exits
        setName("PortfolioMonitor");
    }

    @Override
    public void run() {
        while (running) {
            portfolioService.checkAllThresholds();
            try { Thread.sleep(INTERVAL_MS); }
            catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                running = false; }
        }
    }
}
```

Explanation: `setDaemon(true)` ensures the thread does not prevent JVM shutdown. The `volatile` keyword on `running` guarantees visibility across threads without synchronisation overhead. The `InterruptedException` catch restores the interrupted status and exits the loop cleanly, preventing thread leak on application close.

4. Implementation Highlights

4.1 Observer Pattern — Real-Time Alert System

Justification: The UI layer (PortfolioDashboard) must react to threshold breaches without PortfolioService having any knowledge of Swing components. The Observer pattern decouples the alert source from its consumer — any class implementing AlertListener can register for alerts.

```
// In PortfolioService - fires alert to all registered listeners
private void fireAlert(String message) {
    for (AlertListener listener : alertListeners) {
        listener.onAlert(message);
    }
}

// In PortfolioDashboard - implements AlertListener
@Override
public void onAlert(String message) {
    SwingUtilities.invokeLater(() -> {
        lblAlertBanner.setForeground(RED);
        lblAlertBanner.setText("\u26A0 " + message);
    });
}
```

SwingUtilities.invokeLater() ensures the UI update occurs on the Event Dispatch Thread even when the alert originates from the background monitor thread, preventing thread-safety violations in Swing.

4.2 DAO Pattern — Decoupled Persistence

Justification: Separating the persistence contract (InvestmentDAO interface) from its implementation (FileInvestmentRepository) means the storage mechanism can be changed — from file to database, for example — without any modification to PortfolioService.

```
// DAO interface - the contract
public interface InvestmentDAO {
    void save(List<Investment> investments)
        throws PortfolioPersistenceException;
    List<Investment> load()
        throws PortfolioPersistenceException;
}

// File implementation - ObjectOutputStream serialisation
public void save(List<Investment> investments)
    throws PortfolioPersistenceException {
    try (ObjectOutputStream oos = new ObjectOutputStream(
        new FileOutputStream(filePath))) {
        oos.writeObject(investments);
    } catch (IOException e) {
        throw new PortfolioPersistenceException(
            "Failed to save: " + e.getMessage(), e);
    }
}
```

5. Testing & Error Handling

5.1 Test Cases Considered

#	Test Case	Input / Action	Expected Outcome
1	Add valid Stock	Name: Reliance, Price: 2400, Qty: 10, Ticker: RELI, MarketPrice: 2750	Investment added, return = ₹3500, return% = +14.58%
2	Add valid MutualFund	Axis Bluechip, NAV: 42 → 51, Qty: 100	Investment added, return = ₹900, return% = +21.43%
3	Add valid Bond	Govt Bond, Face ₹1000, Coupon 8%, Qty 5, held 365 days	Return = ₹400, return% = 8.0%
4	Empty name	Name field blank, all other fields valid	InvalidInvestmentException : Name cannot be empty
5	Negative price	purchasePrice = -500	InvalidInvestmentException : Purchase price must be > 0
6	Zero quantity	quantity = 0	InvalidInvestmentException : Quantity must be > 0
7	Non-numeric price	User types 'abc' in price field	NumberFormatException caught in dialog, error shown
8	Invalid date format	purchaseDate = '32-13-2025'	DateTimeParseException caught, error shown
9	Remove non-existent ID	removeInvestment('FAKE-999')	InvalidInvestmentException : ID not found
10	Gain threshold alert	Stock return% >= gainThreshold (default 20%)	onAlert fires, banner updates in UI
11	Loss threshold alert	Stock return% <= lossThreshold (default -10%)	onAlert fires, loss alert shown in red
12	Persist and reload	Save portfolio, restart app, load	All investments reloaded with correct values
13	Corrupted .dat file	Manually corrupt portfolio.dat, load app	PortfolioPersistenceException, empty portfolio loaded
14	Empty portfolio sort	Call getSortedPortfolio() on empty portfolio	Returns empty list, no exception
15	Update non-existent ID	updateInvestment with ID not in portfolio	InvalidInvestmentException : ID not found

Table 3: Test Cases and Expected Outcomes

5.2 Edge Cases Handled

- Zero-invested portfolio: `getTotalReturnPercentage()` guards against division by zero with an explicit check (if `invested == 0` return 0).
- Bond held for 0 days: `ChronoUnit.DAYS.between` returns 0, `calculateReturn()` returns 0.0 — no arithmetic error.
- Data directory missing: `FileInvestmentRepository.save()` calls `file.getParentFile().mkdirs()` before writing to ensure the data/ directory is created if absent.
- No saved data on first launch: `load()` checks `file.exists()` before attempting deserialisation; returns an empty `ArrayList` if no file found — no exception on fresh start.
- Negative loss threshold: `lossThreshold` is stored as a negative double (e.g. -10.0); the check (`ret <= lossThreshold`) handles negative values correctly.
- Thread interrupt during sleep: `InterruptedException` catch restores interrupted status via `Thread.currentThread().interrupt()` and sets `running = false` — clean thread exit.
- Selecting no row before Edit/Delete: UI checks `getSelectedRow() == -1` and shows a dialog prompting selection before proceeding.

6. Git Workflow

The project followed incremental commit discipline across multiple development days. Each major feature and refactoring step was committed separately with a descriptive message.

Commits on Mar 17, 2026		
readme update	ayush-824 committed last week	8dc6687
exception handling update	byayushdas committed last week	d6e84e0
Commits on Mar 18, 2026		
model update	byayushdas committed last week	b4357a0
edit	byayushdas committed last week	5857401
Commits on Mar 19, 2026		
dao update	byayushdas committed last week	42fe57a
Commits on Mar 20, 2026		
service update	byayushdas committed last week	537813f
uml update	byayushdas committed last week	d5aae6e
Commits on Mar 22, 2026		
ui change	byayushdas committed 5 days ago	4f3ae7e
utility update	byayushdas committed 4 days ago	5282821
Commits on Mar 25, 2026		
slides update	byayushdas committed yesterday	b068ea0

Table 4: Git Commit History Summary

7. Conclusion & Future Scope

The Personal Investment Portfolio Tracker demonstrates disciplined object-oriented design across all six required OOP concepts — abstraction, inheritance, polymorphism, exception handling, collections, and threading — within a focused, well-structured codebase. The system is small by design: its value lies not in feature breadth but in the clarity of its architecture, the intentionality of each design decision, and the robustness of its error handling.

Every class has a single responsibility. Every interface has a clear purpose. Every exception carries domain-specific meaning. The layered architecture ensures that changing any one layer — storage mechanism, sort strategy, alert consumer — requires modification in exactly one place.

Future scope for system extension includes:

- Database persistence: replacing `FileInvestmentRepository` with a `JdbcInvestmentRepository` implementing the same `InvestmentDAO` interface — zero service layer changes required.
- Live price integration: adding a `PriceFetchThread` that polls a market data API and updates `currentMarketPrice` on `Stock` objects, triggering automatic portfolio recalculation.
- Additional asset types: adding a new subclass (e.g. ETF, REIT) requires only implementing the four abstract methods — the service, DAO, and UI layers require no modification.
- Export functionality: adding a `CsvExporter` class in a new `export/` package that reads from the `PortfolioService` without coupling to storage or UI logic.

Appendix.

A. Copy of Initial Project Proposal

B. Copy of Approval Email

1. Project Title

Personal Investment Portfolio Tracker

2. Problem Statement

Individual investors managing personal portfolios across multiple asset types ; stocks, mutual funds, and bonds; often rely on disconnected spreadsheets or manual tracking, which lack real-time recalculation and proactive alerts. This leads to poor visibility into actual returns and missed threshold-based decisions. The Personal Investment Portfolio Tracker addresses this by providing a structured, object-oriented system that models each investment type distinctly, calculates returns polymorphically, persists portfolio state across sessions, and notifies users when gain or loss thresholds are breached ; enabling disciplined, informed portfolio management.

3. Target User

Individual retail investors, finance students, and early-career professionals who self-manage small personal investment portfolios without access to professional portfolio management tools.

4. Core Features

- Add, update, and remove investment holdings (Stock, Mutual Fund, Bond)
- Polymorphic return calculation specific to each investment type
- Real-time portfolio summary with total value and net gain/loss
- Threshold-based alerts when gain or loss exceeds a user-defined limit
- Sortable holdings view : by return percentage, current value, or risk level
- Persistent portfolio storage : data saved and reloaded across sessions
- Robust exception handling for invalid inputs and file I/O failures

5. OOP Concepts to be Used

- Abstraction: Abstract Investment class defining shared contract for all asset types
- Inheritance: Specialized subclasses : Stock, MutualFund, Bond : for asset-specific behaviour
- Polymorphism: Overridden calculateReturn() logic per investment subclass
- Exception Handling: Invalid input and portfolio persistence failure handling
- Collections: List<Investment> with Comparator-driven dynamic sorting
- Threads: Background thread for periodic portfolio recalculation and threshold alerts

6. Proposed Architecture Description

The system follows a three-layer object-oriented architecture. The Presentation Layer consists of a Java Swing GUI that displays portfolio summaries and captures user input without containing any business logic. The Service Layer houses the PortfolioService, which coordinates return calculations, threshold evaluations via an AlertManager (Observer pattern), and sorting strategies (Strategy pattern). The Persistence Layer implements a DAO interface with a file-based repository responsible for serializing and deserializing portfolio state across sessions.

Investment (abstract) → Stock / MutualFund / Bond → PortfolioService → AlertManager + SortStrategy → InvestmentDAO → FileRepository

PCCCS495 Proposal Review – APPROVED_MINOR

From soumadip.biswas@ieminternational.ai <soumadip.biswas@ieminternational.ai>

Date Mon 09-03-2026 13:40

To Ayush Das <Ayush.Das2024.1@iem.edu.in>

Dear Ayush Das,

Project: Personal Investment Portfolio Tracker

Decision: APPROVED_MINOR

Score: 90

Strengths:

The proposal presents a clear and relevant problem statement that is well-addressed by the proposed object-oriented solution. It demonstrates an excellent understanding of core OOP principles (Abstraction, Inheritance, Polymorphism) by directly applying them to model different investment types and their calculations. The three-layer architecture is sound, well-structured, and shows a good separation of concerns, effectively delegating responsibilities between the Presentation, Service, and Persistence layers. The inclusion of specific design patterns like Observer (for alerts) and Strategy (for sorting) significantly strengthens the design quality and shows forethought. The feature set is comprehensive, practical, and appropriately scoped for a university-level project, covering essential functionalities like CRUD operations, real-time summaries, persistence, and exception handling.

Improvements:

While 'Threads' is listed as an OOP concept to be applied, its specific use case within the proposed architecture or features is not explicitly detailed. Clarifying where threading would be leveraged (e.g., for background saving, asynchronous alert processing, or purely in the context of the Swing EDT management) would strengthen this aspect. Additionally, for the persistence layer, briefly mentioning the intended file format (e.g., plain Java serialization, JSON, CSV) would add a touch more completeness, though not strictly necessary for this stage.

Regards,
Course Instructor